DATA 430 Project 2 Report: Spark Data Processing and Analytics Pipeline

Nov 8, 2026 · @Nevin

1. Introduction

Our Project 2 pipeline takes 558,746 messy raw records, cleans them with Apache Spark, and loads 512,156 clean rows plus 17 analytics tables into PostgreSQL. Airflow orchestrates the whole run on a daily schedule. A full run at 100,000 records per entity finishes in about three minutes, and all 34 automated validation checks pass.

Problem statement

In Project 1 we built an ETL pipeline that generated retail data with Faker and loaded it into PostgreSQL. The data was clean by construction, and all processing happened in a single Python process. Real data rarely arrives that way: files contain duplicates, missing values, inconsistent spellings and records that point at rows that don't exist. A single process also stops being practical once data outgrows one machine's memory.

Project 2 tackles both problems. We deliberately inject seven kinds of quality problems into the generated data, then move the heavy processing into Spark so the same code would scale from a laptop to a cluster.

Business scenario

We model a mid-sized online retailer that sells about 100,000 products across ten categories to 100,000 customers in three tiers: Regular, Premium and VIP. Each night, exports from the store's systems land as CSV and JSON files. The business wants a trustworthy warehouse it can query the next morning, answering questions such as:

- Who are our most valuable customers, and how much is each tier worth?
- Which products and categories drive revenue?
- How do sales move month to month, and how much do discounts cost us?
- Which categories does each customer tier buy most?

Project goals

1. Generate 100,000 records per entity with realistic, measurable quality problems.

2. Cleanse and deduplicate every entity with PySpark, logging exactly what was dropped or fixed and why.
3. Compute customer, product, order and cross-entity analytics with the Spark DataFrame API and Spark SQL.
4. Load the results into PostgreSQL safely, so a failed load never leaves half a run behind.
5. Orchestrate everything with Airflow, with retries, alerts, validation and an audit trail.
6. Run the whole stack on one machine with a single `docker compose up`.

How this extends Project 1

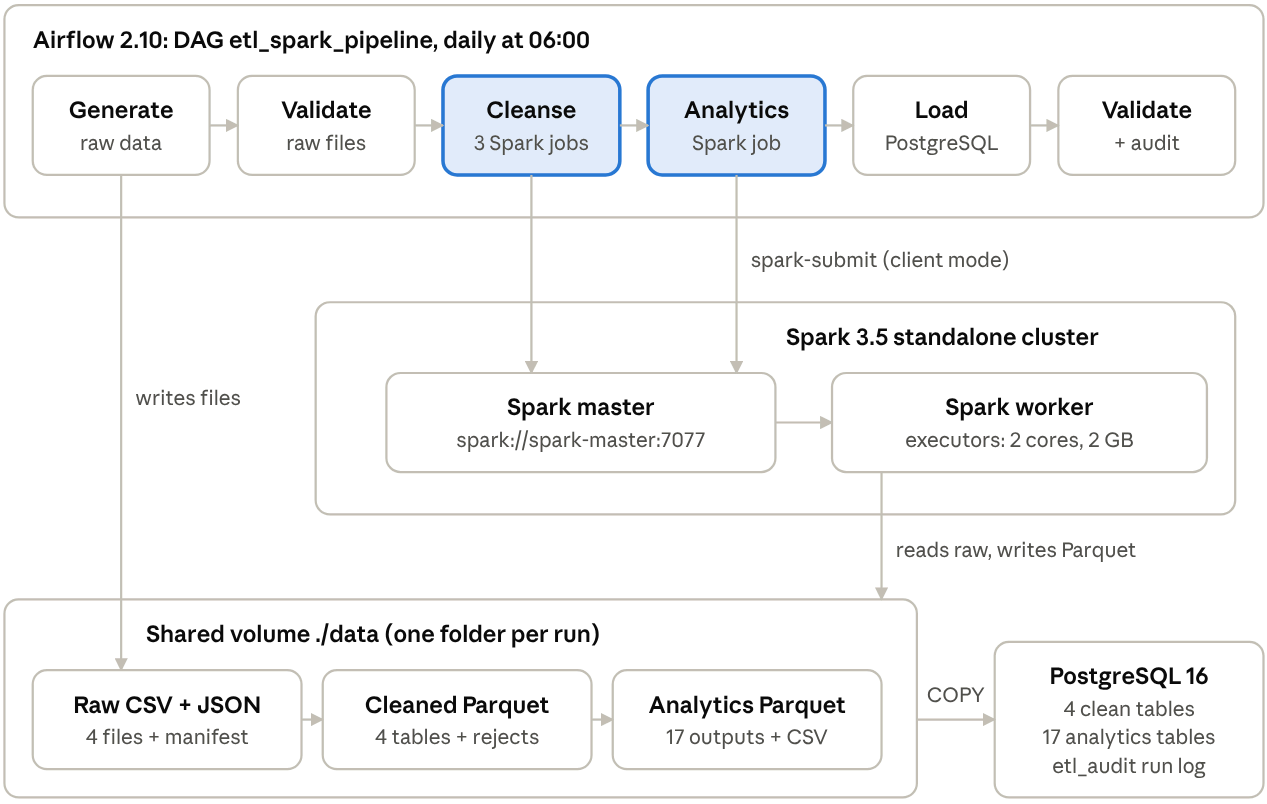
Area	Project 1	Project 2
Processing engine	pandas in one Python process	PySpark on a standalone Spark cluster (master + worker)
Data quality	Clean generated data	7 injected issue types, resolved and counted per reason
Data model	Customers, products, orders	Adds `order_items` linking orders to products, plus an `analytics` schema
Orchestration	Airflow runs Python tasks	Airflow submits Spark jobs with `spark-submit` and runs two in parallel
Reliability	Basic task failure	Retries, failure alerts, transactional loads, 34 validation checks, `etl_audit` table
Deployment	Local setup	Six-container Docker Compose stack, all secrets in `.env`

2. Architecture

Airflow decides what runs and when, Spark does every heavy transformation, and PostgreSQL stores the results. All six services run as Docker containers on one machine.

Airflow orchestrates, Spark processes, PostgreSQL stores

Six Docker containers on one machine. Blue tasks are Spark jobs; their driver runs inside the Airflow scheduler.



System architecture · 6 containers, 9 Airflow tasks

Data moves left to right along the bottom of the diagram. Raw files become cleaned Parquet, then analytics Parquet, then PostgreSQL tables. Airflow never touches the data itself. Its tasks either run a short Python step or submit a Spark job, and Spark's executors read and write the shared `./data` folder directly.

Components

Container	Role
`airflow-webserver`	Web UI on port 8080 for triggering runs and reading task logs
`airflow-scheduler`	Schedules the DAG and runs tasks (LocalExecutor); also hosts the Spark driver for each job

Container	Role
`airflow-init`	One-time setup: database migration and admin user, then it exits
`spark-master`	Accepts submitted applications and assigns executors (UI on port 8081)
`spark-worker`	Runs the executors that do the actual processing
`postgres`	One server, two databases: Airflow's metadata and the `retail_dw` warehouse

How a Spark task runs

1. Airflow's BashOperator runs `spark-submit --master spark://spark-master:7077 --deploy-mode client`.
2. The driver starts inside the scheduler container and asks the master for executors.
3. The master starts executors on the worker. Customers and products each get one core, so they run in parallel.
4. Executors read the raw files and write Parquet to `./data`, which is mounted at the same path in every container.
5. The job exits with code 0 on success or 1 on any error, and Airflow marks the task accordingly.

Task order: `generate_data` → `file_validation` → (`spark_cleanse_customers` ‖ `spark_cleanse_products`) → `spark_cleanse_orders` → `spark_analytics` → `load_to_postgres` → `data_validation` → `completion`.

3. Data Quality

Spark removed 46,590 bad records (8.3% of the raw rows) and repaired values in thousands more, leaving zero duplicates, zero orphans and zero whitespace problems in the warehouse. Every figure below comes from the run of 7 November 2026 (100,000 base records per entity, seed 42).

How we injected the problems

`generate_data.py` first builds clean records, then corrupts a set percentage of them and writes a ground-truth file (`generation_manifest.json`) recording exactly how many of each problem it created. That manifest let us check Spark's results against a known answer instead of guessing.

Issue	What we injected (examples)	How Spark resolved it
Duplicates	2% exact copies of rows; 3% near-duplicates (same ID, one field changed or missing)	Exact copies removed with a window over every column. For near-duplicates we kept the most complete record; for orders, ties went to the most advanced status (Delivered over Pending)
Null values	Missing emails, names, cities, product names, prices, order totals and keys	Null keys dropped. Other fields filled by business rule (`Unknown` , `Regular` , 0). Null order totals were recomputed from the order's line items
Inconsistent formats	`new york` / `NEW YORK` , four phone styles, `$12.99` , `credit_card` / `CREDIT CARD` , `yes` / `Y` / `1`	Title case, `XXX-XXX-XXXX` , decimals, a lookup map for payment methods, real booleans
Invalid values	Negative totals, discounts larger than the order, negative stock, future dates	Invalid orders dropped; stock and prices fixed to 0 or NULL and flagged
Orphan records	Orders for customers that don't exist; line items for missing products	Joined against the cleaned parent tables and dropped
Whitespace	Leading and trailing spaces, tabs, double spaces	Regex trim and collapse (Spark's `trim()` only removes spaces, not tabs)
Date formats	`2024-01-15` , `01/15/2024` and `Jan 15, 2024` mixed in every date field	Tried all three patterns and stored the first match as a proper DATE or TIMESTAMP

Issue	What we injected (examples)	How Spark resolved it
Malformed lines (extra)	Broken CSV and JSON lines	Read in `PERMISSIVE` mode, caught in `_corrupt_record`, dropped and logged

Before and after record counts

Entity	Raw records	Clean records	Dropped	% dropped
Customers	105,064	99,669	5,395	5.1%
Products	105,064	99,714	5,350	5.1%
Orders	105,063	94,356	10,707	10.2%
Order items	243,555	218,417	25,138	10.3%
Total	**558,746**	**512,156**	**46,590**	**8.3%**

Why records were dropped

Reason	Customers	Products	Orders	Order items
Malformed line	4	4	3	2
Exact duplicate	2,078	2,082	2,102	4,799
Near-duplicate (less complete copy)	2,965	2,965	2,858	2,293
Null or invalid primary key	348	299	339	476
Null or invalid foreign key	–	–	433	1,452
Missing, unparseable or impossible date	–	–	1,281	–
Negative total / discount > total	–	–	1,846	–
Missing total, bad quantity or missing price	–	–	16	1,234
Orphan (parent record missing)	–	–	1,829	14,882

The numbers line up with the manifest. For example, we injected 2,060 exact customer duplicates and Spark removed 2,078. The extra 18 are near-duplicates whose "change" happened to be no change, such as blanking a phone number that was already blank.

The orphan counts are higher than what we injected, and that is expected. Order items reference 1,177 injected fake products, but they also lose their parent when a product or an order is itself rejected. The 12,981 items whose order was dropped earlier in the run show how errors cascade through related tables.

Values fixed while keeping the record

Not every problem justified throwing a record away. When Spark repaired a value, it added a label to that row's `dq_flags` column, so anyone querying the warehouse can see what was changed. The largest fixes:

Fix	Records
Date converted to ISO format (all entities)	87,777
City case standardized (`new york` to `New York`)	29,563
Phone number reformatted	24,596
Category case standardized	14,739
Order status case standardized	14,083
Payment method mapped to a standard name	13,885
Customer type case standardized	9,531
Order total recomputed from line items	1,872
Duplicate email removed from a second customer	167

We also kept every dropped record. Each one is saved to `data/rejected/<run>/<entity>/` along with its reject reason, which gave us an audit trail and made debugging much easier.

4. Analytics Results

The cleaned data shows $22.93 million in net revenue from 94,356 orders, an average of $242.96 per order. Three findings stand out: VIP customers are 8% of the base but bring in 21% of revenue, the holiday season nearly triples monthly sales, and Electronics alone earns 30% of revenue.

All 17 outputs are computed in `spark_analytics.py`, about half with the DataFrame API and half with Spark SQL. Each is written as Parquet and CSV and loaded into the `analytics` schema in PostgreSQL. Revenue always means net revenue: the order total minus its discount, excluding shipping.

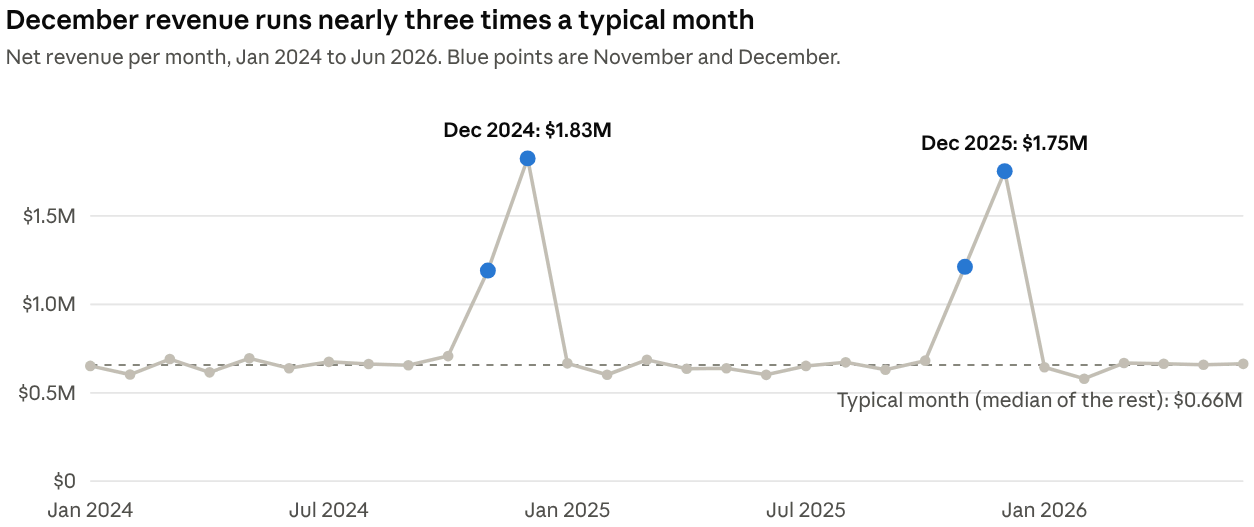
Customer analytics

Customer tier	Share of customers	Share of revenue	Orders	Avg. order value
Regular	70.4%	52.1%	51,030	$234.20
Premium	21.8%	27.0%	26,434	$234.45
VIP	7.8%	20.8%	16,892	$282.76

VIPs earn their place twice over: they order more often and spend about 21% more per order. Nine of the ten highest-value customers are VIPs. The top customer, in San Antonio, placed 44 orders worth $13,114.99. Of the 41,431 customers who ordered at all, 52% came back for a second order, at 2.28 orders per buyer on average.

Rank	Customer	Tier	City	Orders	Lifetime value
1	Dean Bush	VIP	San Antonio	44	$13,114.99
2	Jenna Marsh	VIP	Las Vegas	19	$12,245.37
3	Bryan Flores	VIP	Philadelphia	15	$11,151.98
4	Dorothy Green	VIP	Philadelphia	28	$9,923.56
5	Wendy Cooper	VIP	Phoenix	35	$9,280.00

Order analytics



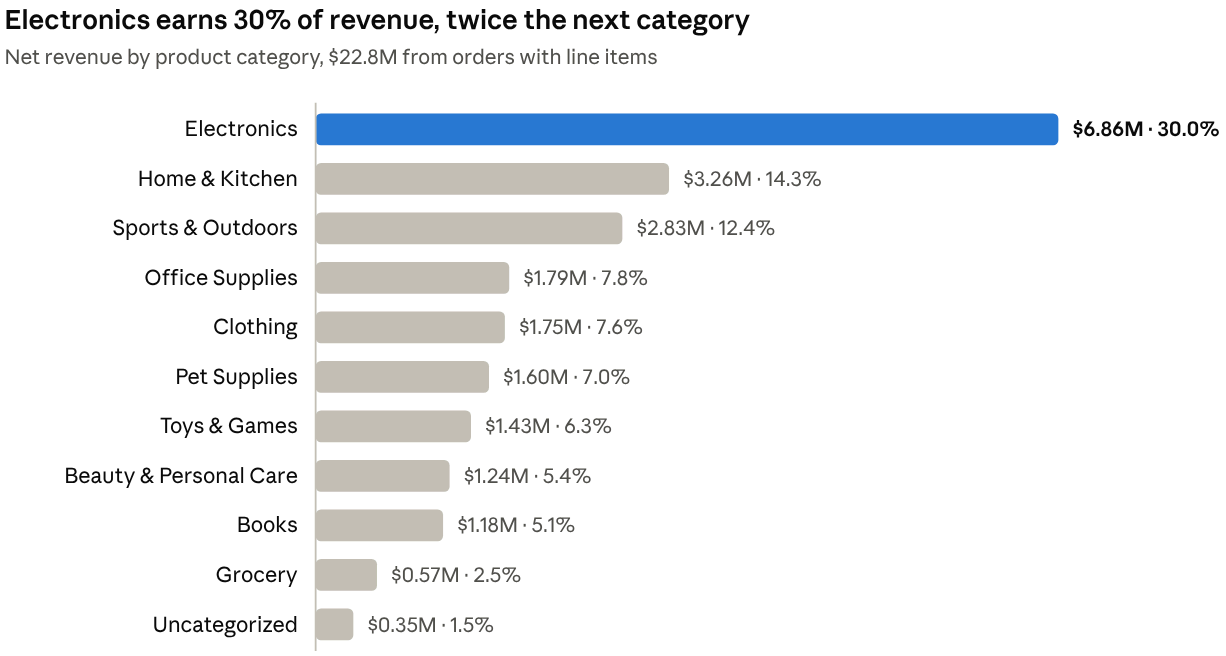
analytics.sales_monthly · run 20261107T150000 · 30 months

Revenue holds steady at about $0.65 million a month, then climbs to $1.2 million in November and $1.8 million in December. That seasonal pattern repeats in both years. Sales are flat across the week: every weekday falls between $3.21 million and $3.38 million.

Payment method	Orders	Share of revenue
Credit Card	35,086	37.0%
Debit Card	20,038	21.1%
PayPal	16,467	17.9%
Apple Pay	11,163	11.8%
Gift Card	4,618	4.9%
Bank Transfer	4,610	4.8%
Unknown (missing in source)	2,374	2.5%

Order status: 87.8% Delivered, 7.6% Shipped, 4.6% Pending. Discounts: 39.2% of orders had one, averaging 14.9% off when applied (5.9% across all orders), for $1.42 million in total discounts.

Product analytics



analytics.category_performance · run 20261107T150000 · 11 categories

Category order counts are nearly even, between 18,900 and 19,900 orders each, so the revenue gap comes from price. Electronics sells for $211 per unit on average against $17 for Grocery. The best-selling single product is a pair of Barnett Deluxe Headphones: 46 units across 30 orders, for $31,349.73. Of the 99,714 products, 88,931 (89.2%) are active, holding 21.9 million units in stock.

Cross-entity analytics

Tier	#1 category	#2 category	#3 category
Regular	Sports & Outdoors	Pet Supplies	Office Supplies
Premium	Office Supplies	Beauty & Personal Care	Sports & Outdoors
VIP	Books	Grocery	Office Supplies

Categories are ranked by number of order lines. The top city by revenue was Tampa ($551,641 from 2,124 orders), but the spread between cities is narrow: the top five fall within 5% of each other.

Because our data is synthetic, some patterns reflect how we built the generator rather than real shopping behavior. We deliberately gave VIPs more orders and larger baskets, and added a holiday bump. The even spread across categories, weekdays and cities comes from random sampling. The analytics found exactly the patterns we built in, which is a useful check that the pipeline computes them correctly.

5. Implementation

The pipeline is seven Python modules, a SQL schema, one Airflow DAG and one Docker image. We kept the Spark jobs and the database code in separate files so each could be tested on its own, outside Airflow.

Modules

Module	Runs as	What it does
`dags/etl_spark_pipeline.py`	Airflow DAG	Defines the 9 tasks, their order, retries, timeouts, callbacks and run parameters (`records` , `seed`)
`scripts/generate_data.py`	BashOperator	Builds 100,000 records per entity with Faker and NumPy, injects the quality problems, writes the ground-truth manifest
`scripts/spark_cleanse.py`	`spark-submit`	Cleanses one entity per call (`--entity customers` , `products` or `orders`); the orders job also cleanses `order_items`
`scripts/spark_analytics.py`	`spark-submit`	Computes the 17 analytics outputs with the DataFrame API and Spark SQL, writing Parquet and CSV
`scripts/spark_utils.py`	Imported by both Spark jobs	SparkSession setup from JSON config, logging, run paths, the statistics tracker, reusable column helpers

Module	Runs as	What it does
`scripts/load_postgres.py`	PythonOperator	Bulk-loads all 21 tables with PostgreSQL `COPY` inside one transaction
`scripts/pipeline_checks.py`	PythonOperator + callbacks	File validation, the 34 warehouse checks, the `etl_audit` record and failure alerts
`sql/schema.sql`	Loader + Postgres init	Clean tables with PK, FK, UNIQUE and CHECK constraints, the `analytics` schema and `etl_audit`

Key design decisions

- **Read everything as text, cast later.** Every raw column is read as a string. Casting happens only after cleaning, so a value like `$12.99` or `N/A` becomes NULL and gets handled instead of crashing the job.
- **Reject or repair, never silently change.** Each rule either drops the record (counted by reason and saved to `data/rejected/`) or fixes it and adds a `dq_flags` label. Nothing changes without a trace.
- **Clean parents before children.** Orders are checked against the *cleaned* customers, and line items against the cleaned products and orders. That is why `spark_cleanse_orders` waits for both parallel jobs, not only customers as the spec's diagram shows.
- **A new `order_items` table.** The spec's order schema has no product column, but product and category analytics need one. Adding a line-item table kept the `orders` columns unchanged.
- **One transaction per load.** The loader truncates and reloads every table inside a single PostgreSQL transaction. We tested this by loading an order for a customer ID that doesn't exist: the foreign key rejected it, everything rolled back, and the previous run's 94,356 orders stayed in place.
- **One image for Airflow and Spark.** Spark runs in client mode, so the driver lives inside the Airflow scheduler container while executors run on the worker. PySpark requires both sides to use the same Python version, so building every service from one image (Python 3.10, Spark 3.5.4, Java 17) removed a whole class of version errors.

- **Every run in its own folders.** Outputs are keyed by the run timestamp (`data/cleaned/20261107T150000/`), so runs never overwrite each other and old results stay comparable.

Challenges and how we solved them

Challenge	Cause	Fix
The orders cleansing job ran out of driver memory	Each "fixed value" flag was appended to one array column, which referenced the previous version twice. Spark's optimizer inlined these, so the expression tree doubled with every flag: about 2^20 nodes after 20 flags	Store each flag in its own column and combine them once at the end with `array_compact`
Still out of memory, this time building the query plan	Each cleaning step was cached, but the plan still carried the full history of every earlier step and grew with each one	Replaced `.cache()` with `localCheckpoint()`, which saves the step's data and cuts the history. All jobs then ran with a 1 GB driver in 20 to 40 seconds
Some values kept their whitespace	Spark's `trim()` removes spaces but not tabs	A regex that collapses any whitespace, then trims
Generator failed inside the Airflow image	Airflow pins NumPy 1.26, which can't join string arrays with `+`	Converted the arrays to Python objects before joining
Two parallel Spark jobs could block each other	The first job took both worker cores, so the second waited for it to finish	Each parallel job requests half the worker's cores, so both run side by side
Rejected-record files lost their column names	Casting a column inside `to_json(struct(...))` drops its name	Re-applied the original name with `.alias()`
Docker build failed with an input/output error on Windows	Docker Desktop's image storage hit a write error while unpacking the Airflow base image, with little free disk space	Freed space on the drive, restarted Docker Desktop and WSL, and cleared the build cache, then rebuilt

Error handling and monitoring

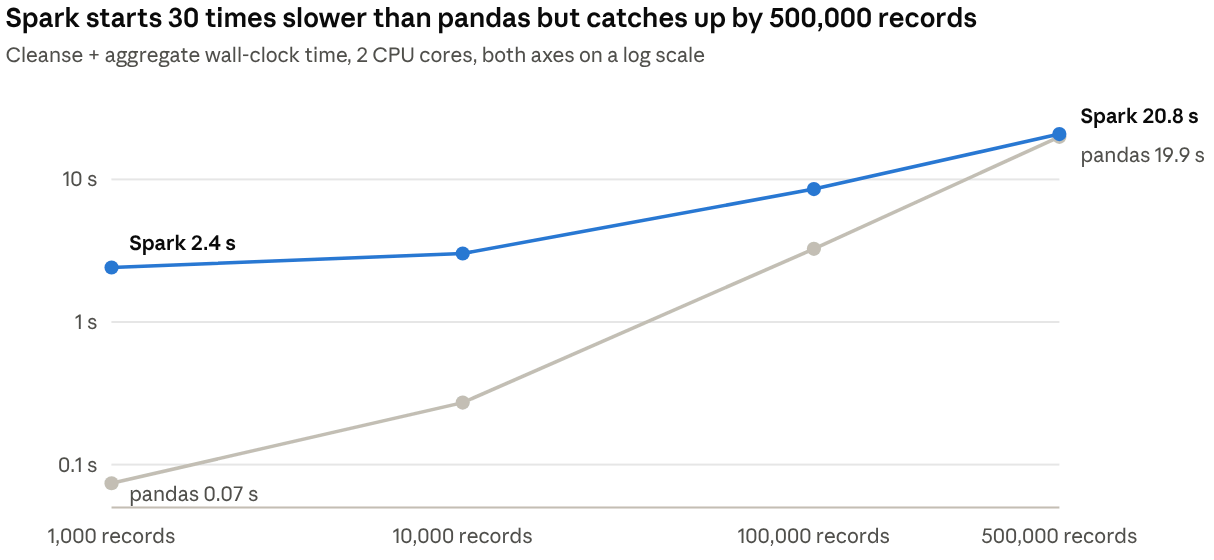
Airflow retries each task once with exponential backoff (twice for the database load), and each task has a 30-minute timeout. A task that fails for good triggers an alert callback. It always logs, and it posts to Slack or sends email when those are set in `.env`. A DAG-level callback writes a `failed` row to `etl_audit` naming the failed task. Every successful run writes a `success` row with start and end times, duration and the raw, clean, rejected and loaded record counts.

6. Comparison: Spark vs. pandas

At our data sizes, pandas was faster. Spark only drew level at 500,000 records, but its run time grew far more slowly. That makes Spark the right choice for data that keeps growing, not for a 100,000-row job on one laptop.

Performance

We wrote `benchmark_pandas_vs_spark.py` to run the same workload in both engines: read customers and orders, normalize strings, remove duplicates, parse the three date formats, drop invalid and orphan orders, then compute lifetime value and monthly revenue. Spark ran in local mode on 2 CPU cores. We gave it one untimed warm-up run and left its 3-second session startup out of the totals.



benchmark_pandas_vs_spark.py · local[*], 2 cores · best of 1 run per size

Spark pays a fixed cost of about 2.4 seconds on every job, for planning the query, scheduling tasks and moving data between the JVM and Python. That overhead swamps the actual work at 1,000 or 10,000 rows. As the data grows, the picture changes. Going from 100,000 to 500,000 records multiplied pandas' time by 6.1, but Spark's by only 2.4. At 500,000 records Spark was already reading the files faster than pandas (4.1 s against 6.1 s), because it reads in parallel.

The extrapolation matters more than the crossover point. pandas has to hold all the data in one machine's memory, so beyond a few million rows it slows sharply or fails. Spark spreads the same work across more cores or more machines without any code changes. Both engines also produced almost the same output: 464,399 clean orders from pandas against 464,394 from Spark at 500,000 records. The small gap comes from how each library treats a few edge-case lines and dates.

Code complexity

	pandas (Project 1 style)	PySpark (Project 2)
Feel	Eager: each line runs at once and you can print the result	Lazy: you build a plan and nothing runs until an action like `count()` or `write`
Debugging	Easy, step by step in a notebook	Harder: errors appear at the action, often as long JVM stack traces
Hidden costs	Memory use is visible and grows with the data	Plan size matters: our two out-of-memory bugs came from plans, not data
Joins and windows	Simple for small data	Need thought about broadcast versus shuffle joins
Setup	`pip install pandas`	Java, a cluster, matching Python versions and network settings
Our benchmark code	About 50 lines	About 50 lines

For the same logic, the line counts were nearly identical. The extra complexity of Spark lives in everything around the code: the cluster, the configuration, and having to think about how the work is planned.

Scalability

pandas scales up: the only fix for more data is a machine with more memory. Spark scales out. Our pipeline already runs on a master and a worker, so handling ten times more data means adding workers and raising `SPARK_WORKER_CORES` in `.env`. The code stays the same. Spark also gives us fault tolerance, since a failed task is retried on another executor, and its web UI lets us see where each job spends its time.

7. Lessons Learned

The biggest lesson was that in Spark, the shape of the query plan matters as much as the size of the data. Both of our serious bugs ran out of memory on fewer than 250,000 rows, and neither had anything to do with data volume.

What worked well

- **Known answers.** Because the generator writes a manifest of exactly what it injected, we could check each Spark result against a known number instead of eyeballing the output.
- **Counting every drop by reason.** The statistics tracker made it obvious when a rule was too aggressive or too weak. It also made the cascade effect visible: 12,981 line items lost their parent order.
- **Letting Airflow orchestrate and Spark compute.** The DAG stayed short and readable. Each Spark job could also be run and debugged by hand with the same `spark-submit` command Airflow uses.
- **The database as a safety net.** Primary keys, foreign keys and CHECK constraints, combined with a single transaction, meant a bad load could never leave the warehouse half-updated.
- **Reconciliation checks.** Testing that customer lifetime value, monthly sales and revenue by customer type all add up to total order revenue, to the cent, gave us real confidence that the analytics were correct.

What we would do differently

- **Look at the query plan early.** We would run `explain()` on each job as we built it, instead of finding plan problems through out-of-memory errors.

- **Use a table format with history.** Writing outputs as Delta Lake or Apache Iceberg tables would give us versioned data and incremental loads, instead of a separate folder per run and a full reload each time.
- **Unit-test the cleansing rules.** Testing each rule on a tiny hand-written DataFrame would catch mistakes in seconds, without generating 100,000 records first.
- **Clean up old runs.** Each 100,000-record run adds about 100 MB to `data/`. A retention task at the end of the DAG would keep only the last few runs.
- **Load incrementally.** Truncating and reloading all 21 tables is simple and safe at this size, but at production scale we would merge only new and changed rows.

Key takeaways

- **Distributed computing has a fixed cost.** Spark loses to pandas on small data and wins as data grows. The right tool depends on where the data is heading, not just where it is today.
- **Lazy evaluation changes how you debug.** Errors show up at the action that runs the plan, not at the line that caused them. Caching, checkpointing and reading plans are core skills, not optimizations to add at the end.
- **Orchestration is about failure, not the happy path.** Most of the DAG's value is in retries, timeouts, alerts, validation and the audit trail, which together tell you what went wrong at 6 a.m. without anyone watching.
- **Environment consistency is part of the design.** Matching Python, Java and Spark versions between the driver and executors, and pinning packages to Airflow's constraints, prevented errors that would have been hard to trace.
- **Data quality is a set of business decisions.** Choosing whether to drop or repair a record, such as rebuilding a missing order total from its line items instead of discarding the order, needed business judgment, not just code.

Appendices

Appendix A: Airflow tasks

#	Task	Operator	What it does
1	`generate_data`	BashOperator	Runs the Faker generator into `data/raw/<run>/`

#	Task	Operator	What it does
2	`file_validation`	PythonOperator	Checks each raw file exists, is readable and non-empty, and has the expected header
3	`spark_cleanse_customers`	BashOperator (`spark-submit`)	Cleanses customers; runs in parallel with task 4
4	`spark_cleanse_products`	BashOperator (`spark-submit`)	Cleanses products
5	`spark_cleanse_orders`	BashOperator (`spark-submit`)	Cleanses orders and order items; waits for tasks 3 and 4
6	`spark_analytics`	BashOperator (`spark-submit`)	Computes the 17 analytics outputs
7	`load_to_postgres`	PythonOperator	Loads 21 tables in one transaction with `COPY`
8	`data_validation`	PythonOperator	Runs the 34 checks in Appendix C
9	`completion`	PythonOperator	Writes the `success` row to `etl_audit` and logs a summary

Schedule: daily at 06:00 America/New_York, `catchup=False`, one active run at a time.

Appendix B: Analytics tables (schema `analytics`)

Table	Rows	Contents
`customer_metrics`	99,669	Lifetime value, order count, average order value, first and last order per customer
`customer_type_distribution`	3	Customers and share per tier
`top_customers`	10	Highest lifetime value
`product_metrics`	99,714	Revenue, orders, units and average selling price per product

Table	Rows	Contents
`category_performance`	11	Revenue, units, orders and share per category
`inventory_status`	2	Active vs inactive products and stock
`sales_monthly`	30	Orders, revenue, AOV and discount per month
`sales_daily`	912	Orders and revenue per day
`order_status_breakdown`	3	Orders and revenue per status
`payment_method_distribution`	7	Orders and revenue share per payment method
`discount_summary`	1	Discount frequency and average rate
`revenue_by_customer_type`	3	Revenue, orders and AOV per tier
`top_categories_by_customer_type`	9	Top 3 categories for each tier
`city_revenue`	46	Customers, orders and revenue per city
`brand_performance`	301	Revenue rank per brand (extra)
`sales_by_weekday`	7	Orders and revenue per weekday (extra)
`repeat_customers`	1	Repeat-purchase rate (extra)

Appendix C: Data validation checks (34 of 34 passed)

- **Row counts (21 checks):** each clean and analytics table in PostgreSQL holds exactly the number of rows Spark wrote.
- **Referential integrity (3):** no order without a customer, no line item without an order, no line item without a product.
- **Business rules (5):** no duplicate emails, no negative amounts, no discount above the order total, no orders dated in the future, no leading or trailing spaces.
- **Reconciliation (5):** customer lifetime value, monthly revenue and revenue by customer type each equal total order revenue within $0.01; monthly order counts equal the number of orders; category revenue equals revenue from orders that have line items within $1.00.

Appendix D: Running the project

```
copy .env.example .env      # set passwords and the secret key
docker compose build        # first time only
docker compose up -d
# Airflow: http://localhost:8080   Spark: http://localhost:8081
docker compose exec airflow-scheduler airflow dags trigger etl_spark_pipeline
docker compose exec postgres psql -U data430 -d retail_dw
```

Full setup and troubleshooting steps are in the repository's `README.md`.